Zero-Cost Loops: Unrolling Variadic Templates for Maximum Performance

How `std::index_sequence` and meta-programming turn runtime loops into compile-time straight-line code.

5 min read · 1 day ago

Sagar Following

Listen Share More

I use loops, you use loops, everybody uses loops — it's the default way we write C++. But the problem with loops is that they are sneakily expensive.

When you write a `for` loop, the CPU has to do work *besides* your actual logic:

1. Check the loop counter
2. Compare it to the limit

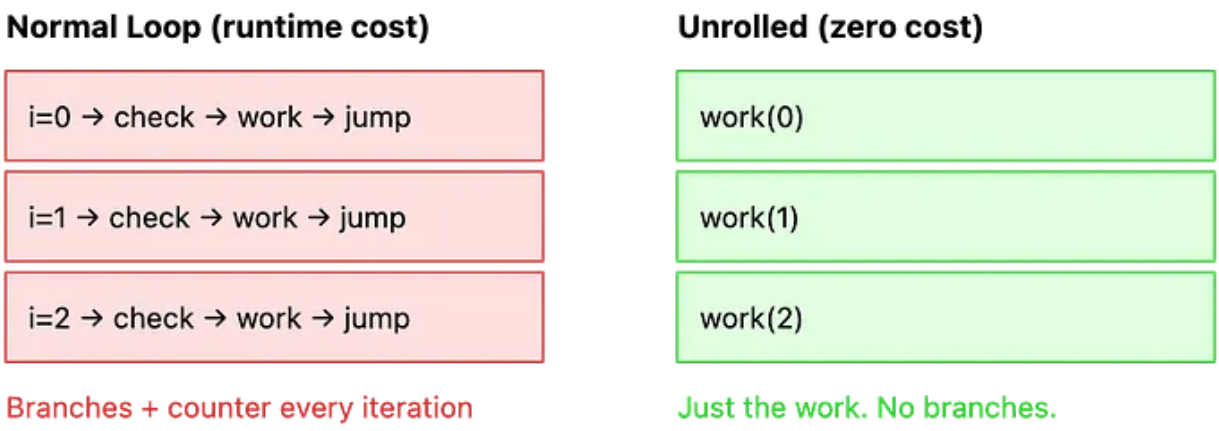
3. Jump back to the top
4. Predict the branch (and sometimes guess wrong)

For a hot path running **millions of times per second**, this overhead adds up. Wouldn't it be nice if the loop just… didn't exist at runtime?

That's what **variadic template unrolling** does. The compiler **"unrolls"** the loop at compile time, leaving behind only your real work.

The Big Idea (in pictures)

Here's what a normal loop looks like vs. an unrolled one:



The compiler literally writes out each call for you. No loop counter exists. No branch exists. The CPU just runs straight through.

Variadic template unrolling uses `std::index_sequence` to generate zero-runtime-cost loops at compile time, eliminating branch prediction misses and enabling **perfect forwarding** in generic hot paths.

Let me show you how.

Step 1: `std::index_sequence`

This is the magic ingredient. It's a type that holds a list of compile-time integers:

```
#include <utility>

// std::index_sequence<0, 1, 2, 3> is a TYPE that carries indices.
// std::make_index_sequence<4> creates that type for you.
```

Think of it as a way to “smuggle” numbers into the type system, where the compiler can do whatever it wants with them — including unrolling.

Step 2: The Unroll Pattern

Here’s the canonical trick. We use a fold expression (C++17) over a parameter pack:

```
#include <utility>
#include <cstddef>

template <typename F, std::size_t... Is>
constexpr void unroll_impl(F&& f, std::index_sequence<Is...>) {
    // Fold expression: expands to f(0); f(1); f(2); ... at compile time.
    (f(std::integral_constant<std::size_t, Is>{}), ...);
}

template <std::size_t N, typename F>
constexpr void unroll(F&& f) {
    unroll_impl(std::forward<F>(f), std::make_index_sequence<N>{});
}
```

So What’s happening here?

- `std::make_index_sequence<N>` creates `index_sequence<0, 1, ..., N-1>`.
- When `unroll_impl` deduces `Is...`, it has every index as a separate compile-time value.
- The fold `(f(...), ...)` expands into one call per index — no loop in sight.
- We pass `integral_constant` so the index is usable as a `constexpr` inside `f`.

Step 3: Let’s Use It

```
#include <array>
#include <iostream>
```

```
int main() {
    std::array<int, 4> data{10, 20, 30, 40};
    int sum = 0;

    unroll<4>([&](auto I) {
        // I is a compile-time constant! The index becomes a
        // compile-time constant — the load from data is still runtime
        // (unless data itself is constexpr), but the offset is baked in.
        sum += data[I];
    });

    std::cout << sum << '\n'; // 100
}
```

The lambda receives `I` as a `std::integral_constant`, which has an implicit `constexpr` value. That means `data[I]` uses an offset known at compile time — the compiler can fold it into the instruction's addressing mode instead of computing `base + i*sizeof(int)` at runtime.

Notice `F&&` and `std::forward<F>(f)`. This is perfect forwarding — it preserves whether you passed an lvalue, rvalue, const, etc. Without it, you might:

- Make unnecessary copies of expensive callables
- Lose `const`-correctness
- Break with move-only types (like a lambda capturing a `unique_ptr`)

```
// Bad: copies f
template <typename F> void unroll_bad(F f) { /* ... */ }

// Good: forwards exactly what the caller gave us
template <typename F> void unroll_good(F&& f) {
    /* uses std::forward<F>(f) */
}
```

Here is the fun part — Look at the Assembly

This is the payoff. With **-O2**, the compiler produces something like:

```
; unroll<4>([](auto I){ sum += data[I]; })
mov     eax, [data+0]      ; sum += data[0]
add     eax, [data+4]      ; sum += data[1]
add     eax, [data+8]      ; sum += data[2]
add     eax, [data+12]     ; sum += data[3]
```

The index becomes a compile-time constant, enabling the compiler to fold address computation into a single load instruction.

Zero branches. Zero counter. Zero loop overhead (reduces to straight-line code with no visible loop structure). Often the optimizer goes further and **SIMD-vectorizes** the whole thing.

"But wouldn't `-O3` auto-unroll this anyway?"

Fair question — and yes, often it would. Modern compilers (GCC, Clang, MSVC) are aggressive about unrolling simple loops with known trip counts. So for plain numeric loops, the assembly might end up identical.

But template unrolling gives you two things **`-O3`** ***cannot***:

1. **A guarantee.** Auto-unrolling is a heuristic. Change the surrounding code, bump the loop bound, or hit a cost threshold and the optimizer may quietly back off. Template unrolling is part of the language semantics — it always happens.
2. **Heterogeneous compile-time dispatch.** A runtime loop body must have *one* type. Template unrolling lets each "iteration" be a different type, different overload, different **`if constexpr`** branch. Things like **`std::get<I>(tuple)`**, per-

field serialization, or static dispatch tables are simply impossible with an ordinary loop, no matter the optimization level.

So: rely on **`-O3`** for arithmetic kernels, reach for template unrolling when you need *correctness* of expansion or ***type-varying*** iteration.

An Example showcasing when `-O3 can't help you` — Iterating a Tuple

The classic real-world use case — and the one `-O3` *can't* help you with:

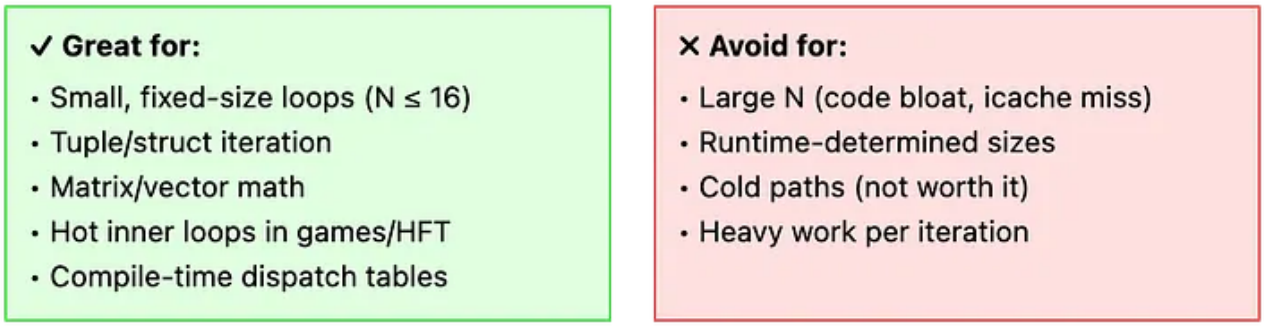
```
#include <tuple>
#include <iostream>

template <typename Tuple, typename F>
constexpr void for_each_tuple(Tuple&& t, F&& f) {
    constexpr auto N = std::tuple_size_v<std::decay_t<Tuple>>;
    unroll<N>([&](auto I) {
        f(std::get<I>(std::forward<Tuple>(t)));
    });
}

int main() {
    auto t = std::make_tuple(1, 3.14, "hello");
    for_each_tuple(t, [](auto&& x) { std::cout << x << ' '; });
    // prints: 1 3.14 hello
}
```

Each **`std::get<I>`** returns a ***different type***, which is impossible with a runtime loop. Variadic unrolling makes it trivial.

When To Use This?



Warning! Variadic unrolling can:

- explode binary size
- hurt instruction cache
- increase compile time

A quick reference about the machinery behind this mechanism

Technique	What it gives you
`std::index_sequence`	Compile-time list of indices
Fold expression `(..., ...)`	Expands one statement per index
Perfect forwarding `F&&`	No copies, works with any callable
`std::integral_constant` index	Index becomes a compile-time constant inside the lambda

Together: **a loop that doesn't exist at runtime.** The compiler emits straight-line code, the branch predictor stays happy, and your hot path runs as fast as physics allows.

Now go open Compiler Explorer and stare at the assembly. It's beautiful.

Closing Thoughts

Variadic unrolling isn't about outsmarting the optimizer — it's about expressing intent the optimizer can't infer. When you write `unroll<N>`, you're telling both the